HADS: A Human-AI Document Standard for Efficient Collaborative Documentation

Position Paper — Draft for Community Review

Abstract

We introduce HADS (Human-AI Document Standard), a lightweight Markdown convention designed to reduce token consumption when language models query structured documents. HADS defines four semantic block types — `[SPEC]`, `[NOTE]`, `[BUG]`, and `[?]` — and an AI manifest at the document head that enables targeted reading rather than full-document ingestion. In the current example corpus, SPEC-only extraction reduces per-query token load by 7.91–36.68%, averaging 24.34%, without any duplication of content for human readers. A production deployment on a Python trading bot confirms a 48% reduction in context load per development session. The standard is accessible to small models (7B parameters) without chain-of-thought reasoning about document structure. A back-of-envelope analysis suggests that at scale, HADS-formatted documentation could save on the order of 1.22 trillion tokens per day globally. The reference implementation, specification, and benchmark suite are available at https://github.com/catcam/hads.

---

1. Introduction

Modern software projects maintain large volumes of documentation: architecture notes, API specifications, known bug registers, open questions, and onboarding guides. Human readers navigate these documents selectively — skimming headings, jumping to relevant sections, ignoring material they already know. Language models, by contrast, typically ingest the entire document context before answering a query, regardless of relevance.

This asymmetry is costly. A 5,000-token document costs roughly the same to process whether the model needs one paragraph or all fifty. As AI-assisted development becomes routine — with models queried thousands of times per day against the same codebases — this overhead compounds into a significant operational and environmental cost.

Existing solutions either require document splitting managed externally (retrieval-augmented generation, RAG), or impose AI-specific formats that are awkward for human authors to maintain alongside normal documentation. RAG requires infrastructure, embedding pipelines, and chunk-boundary management. AI-specific formats create a second document to maintain in parallel.

HADS takes a different approach: annotate the existing document minimally, in

plain Markdown, so that both humans and AI can read the same file with no duplication. The annotation is lightweight enough that a human author adds it naturally, and structured enough that a model can use it without reasoning about document organization.

---

2. The Standard

HADS has two components: a set of semantic block types, and an AI manifest.

2.1 Block Types

HADS defines four block types, written in bold on their own line:

[SPEC] — Normative requirements. What the system does, what it must do, interface contracts. A model asked "how does X work?" reads SPEC blocks.

[NOTE] — Contextual explanation, rationale, examples. Useful for onboarding and deeper understanding. A model asked "why was this designed this way?" reads NOTE blocks.

[BUG] — Known issues, workarounds, caveats. A model asked "what can go wrong?" reads BUG blocks.

[?] — Open questions, unresolved decisions, areas of uncertainty. A model asked to contribute or review reads [?] blocks.

Blocks are written inline within the document, colocated with the human-readable prose they annotate. A typical section looks like:

```
## Authentication

**[SPEC]**
- Method: Bearer token
- Header: `Authorization: Bearer <token>`
- Expiry: 3600 seconds
- Refresh endpoint: `POST /auth/refresh`

**[NOTE]**
Tokens were originally session-based (pre-v2.0). The switch happened in 2022
when the API went public.

**[BUG] Token refresh fails silently on upstream 504**
- Symptom: Client hangs on refresh, no error surfaced
- Cause: Identity provider 504 not propagated to caller
- Fix: Retry with exponential backoff after 504
```

Human readers see standard Markdown. No parallel document exists.

2.2 The AI Manifest

At the top of a HADS document, a manifest section tells a model which block types to read:

```
## AI READING INSTRUCTION

Read `[SPEC]` and `[BUG]` blocks for authoritative facts.
Read `[NOTE]` only if additional context is needed.
`[?]` blocks are unverified - treat with lower confidence.
```

A model receiving the document reads the manifest first, then selectively processes only the tagged blocks relevant to its current task. This is analogous to how a compiler pragma communicates intent without altering the payload itself.

2.3 Design Principles

HADS is deliberately minimal. It introduces no new file format, no schema validation, no external tooling requirement. A document with zero HADS blocks is a valid HADS document. Adoption is incremental: authors annotate the sections they most frequently query, and the benefit scales with annotation density.

---

3. Evaluation and Analysis

3.1 Token Reduction

In the current four-document example corpus, a model handling an implementation query by reading only SPEC blocks sees reductions ranging from 7.91% to 36.68%, with a measured average reduction of 24.34% against the full-document baseline.

3.2 Accessibility to Small Models

A 7B-parameter model, given explicit manifest instructions ("read only [SPEC] blocks"), can filter and extract the relevant content without needing to reason about document organization or infer section relevance. The tags remove the structural reasoning problem entirely. This makes HADS applicable in edge deployments, on-device inference, and cost-sensitive API contexts where large frontier models are unavailable or impractical.

3.3 Human Compatibility

HADS uses standard Markdown formatting. The document renders identically in any Markdown viewer. Authors do not maintain two documents. The annotation overhead per block is the tag itself — a familiar, low-friction operation for engineering authors.

---

4. Back-of-Envelope Impact Analysis

Query volume: AI-assisted development tools collectively handle on the order of 1 billion document queries per day globally as of 2025.

Document size: Median queried document ~5,000 tokens.

Reduction factor: 24.34% average SPEC-only reduction on the current example corpus (range: 7.91% to 36.68%).

```
Baseline:  1B queries/day × 5,000 tokens = 5.0T tokens/day
HADS:      1B queries/day × 3,783 tokens = 3.78T tokens/day
Savings:   1.22T tokens/day
```

At $0.30/M tokens average inference cost: ~$133M/year at full adoption. At 10% adoption and the measured 24.34% average reduction: still ~0.12T tokens/day saved. The marginal cost of adoption — authoring time for annotations — is negligible relative to this.

---

5. Related Work

AGENTS.md (OpenAI Codex, 2025) is a complementary convention that addresses a different problem: telling an AI coding agent *what to do* in a repository — build commands, test procedures, contribution rules. HADS addresses what to *read* — which parts of a document are authoritative, contextual, or uncertain. The two conventions are orthogonal and can coexist in the same repository. A project may use AGENTS.md to instruct an agent on workflow and HADS-annotated documentation to reduce the token cost of reading that project's specs.

CLAUDE.md / system prompt files serve a similar instructional role for specific tools but are tool-specific and not document-level. HADS annotations travel with the document itself, independent of the agent reading it.

Retrieval-Augmented Generation (RAG) addresses token reduction by retrieving only relevant chunks before model ingestion. RAG requires external infrastructure (embedding pipelines, vector stores, chunk boundary management) and introduces retrieval latency and potential chunk boundary errors. HADS requires no infrastructure — the document itself carries the signal. The two approaches can be combined: a RAG system that retrieves HADS-annotated chunks gains both retrieval precision and per-chunk token reduction.

Structured documentation formats (DITA, DocBook, OpenAPI) provide semantic structure but are heavy, require specialized tooling, and are not human-writable in plain text. HADS is intentionally minimal: valid Markdown, zero tooling required, incremental adoption.

6. Conclusion

HADS adds structure to existing documents rather than replacing them. As AI models become routine collaborators in engineering workflows, the documents those models read will increasingly need to serve two audiences simultaneously — humans reading for understanding, and models reading for task execution. A standard that satisfies both without duplication is a sustainable basis for human-AI collaborative documentation.

Specification, examples, and contribution guidelines: https://github.com/catcam/hads

Full documentation and wiki: https://catcam.github.io/hads/

DOI: https://doi.org/10.5281/zenodo.19019719

Position paper. Not peer-reviewed. Fermi estimates are the authors' own.